

MSc in Data Science

Data Analytics & Algorithms — Continuous Assessment 1

FaceTrace

An AI-Powered Reverse Face Search System

Comparative implementation of CNN and Vision Transformer architectures, in NumPy.

Field	Detail
Module	Data Analytics & Algorithms
Assessment	CA1 (100% of module mark)
Group Members	Anjaly · Surekha

Table of Contents

1. Product Definition & Sustainability.....	3
1.1 Product Definition.....	3
1.2 Sustainability & Resilience Objectives.....	3
1.3 Feasibility & Scope.....	4
2. Data Acquisition & Preparation.....	5
2.1 Dataset Selection, Sourcing, Ethics & Compliance.....	5
2.2 Exploratory Data Analysis.....	5
2.3 Data Cleaning & Feature Engineering.....	6
2.4 Justification of Pre-processing.....	6
3. Product Development and Algorithms.....	8
3.1 Algorithm Selection.....	8
3.2 Implementation & Tuning.....	8
3.2.1 CNN Architecture.....	8
3.2.2 ViT Architecture.....	9
3.2.3 Similarity Search.....	10
3.3 Iterative Product Development.....	10
3.4 Integration of Sustainability & Resilience.....	11
4. System Architecture & Deployment.....	12
4.1 End-to-End Pipeline.....	12
4.2 Prototype / Demo.....	13
4.3 Usability.....	13
5. Critical Evaluation & Limitations.....	14
5.1 Algorithm Comparison Results.....	14
5.2 What Worked.....	14
5.3 Limitations.....	15
6. Why Internet-Wide Search Was Not Achievable.....	16
6.1 What PimEyes Does.....	16
6.2 Why Our Attempt Failed.....	16
6.3 Pivot.....	16
7. Conclusion.....	18
8. References.....	19
Appendix A. Individual Contribution Summary.....	20

1. Product Definition & Sustainability

1.1 Product Definition

FaceTrace is a privacy-preserving reverse face search web application. A user uploads any photograph containing a human face, and the system returns the most visually similar faces from a locally indexed image database, ranked by cosine similarity. The product targets two user-facing scenarios: (i) lawful identity verification by accredited bodies who already have legal authority to maintain a face database (border control, missing-persons units, regulated KYC providers); and (ii) self-investigative use by individuals wishing to discover whether their own image appears in a curated dataset they control. FaceTrace is the local, auditable, open-source counterpart to commercial services such as PimEyes, designed to operate without uploading biometric data to any third party.

The product implements and compares two distinct neural network architectures, both built from scratch using only NumPy — no machine learning framework (TensorFlow, PyTorch, JAX) was used. Algorithm 1 is a four-block Convolutional Neural Network (CNN) that processes faces through hierarchical sliding-filter operations, extracting local spatial features. Algorithm 2 is a patch-based Vision Transformer (ViT) (Dosovitskiy et al., 2020) that splits each face into 64 non-overlapping 8×8 patches and uses multi-head self-attention to model global relationships across the entire face. Both algorithms output 128-dimensional L2-normalised embeddings that are compared using cosine similarity, allowing direct, controlled head-to-head comparison through a shared comparison interface.

1.2 Sustainability & Resilience Objectives

FaceTrace addresses sustainability and resilience along three explicit axes: environmental sustainability, social responsibility, and system resilience. Each is operationalised in the design rather than asserted as a slogan.

Environmental sustainability — Inference is performed on commodity CPUs without GPU acceleration. End-to-end search latency is approximately 200ms (CNN) and 800ms (ViT) on a standard Intel i5 laptop. The CNN model has approximately 500K parameters; ViT has approximately 2M. Compared with cloud-hosted face recognition services that depend on GPU clusters, the per-query carbon footprint is several orders of magnitude lower. The 64×64 input resolution was chosen partly for accuracy and partly to keep memory and energy usage within laptop budgets.

Social responsibility — The system processes images entirely on the user's device. No images, embeddings, or query payloads are transmitted to any external server. This is structurally enforced: the application binds only to localhost, and there is no network egress in the search path. This

design choice eliminates the privacy risk associated with cloud-based facial recognition services and addresses GDPR Article 9 concerns regarding biometric data. The scope is also explicitly bounded — Section 4 documents why we did not extend FaceTrace into an internet-wide crawler, even when this was technically tempting.

System resilience — The application functions fully without internet access after initial setup. The technology stack (Flask, OpenCV, NumPy) consists exclusively of open-source components with no vendor lock-in. The image database is stored as plain NumPy .npy arrays plus JSON metadata, which is auditable, portable, and not coupled to any proprietary database engine. If a single component fails (e.g. ViT server crashes), the CNN service remains independently operational on a separate port, and vice versa.

1.3 Feasibility & Scope

The Labeled Faces in the Wild (LFW) Top-50 subset was selected as a computationally feasible benchmark for an academic timeframe of approximately eleven weeks. Both architectures were successfully implemented from scratch and deployed as functional web applications. A comparison page demonstrates the practical, observable difference between local convolutional features (CNN) and global attention-based features (ViT) for face recognition. Out of scope are: training the networks on labelled face identity data (random initial weights are retained intentionally to keep the comparison architectural rather than data-driven); building a GPU training pipeline; and crawling external image sources. The reasons these were excluded are documented in Section 4.

2. Data Acquisition & Preparation

2.1 Dataset Selection, Sourcing, Ethics & Compliance

The Labeled Faces in the Wild (LFW) dataset (Huang et al., 2007) was selected as the primary data source. LFW is a publicly available benchmark dataset maintained by the University of Massachusetts Amherst, created specifically for the study of unconstrained face recognition. The full dataset contains 13,233 images of 5,749 individuals at 250×250 pixels in JPEG format. The license explicitly permits academic and non-commercial use. Subjects are public figures whose images were sourced from news photography that was already in the public domain at the time of collection. We used the LFW Top-50 subset, comprising the 50 most-photographed individuals; this provides enough images per identity to demonstrate ranked retrieval while remaining tractable on a laptop.

Ethical and compliance considerations were treated as gating requirements rather than afterthoughts. Three checks were applied. First, the dataset license was reviewed before download. Second, an explicit decision was taken not to enrich the dataset with images scraped from the live web; this is documented in Section 4 alongside the GDPR analysis. Third, the application contains no real-time surveillance capability — there is no streaming-from-webcam endpoint and no API to ingest images from arbitrary remote URLs. The product is consistent with the GDPR principle of data minimisation: only what is needed to demonstrate a face search workflow is processed, and processing is local.

2.2 Exploratory Data Analysis

Four EDA findings shaped the rest of the project.

Property	Finding	Project impact
Image distribution	Heavily skewed long-tail. George W. Bush dominates with 530 images; the median identity has fewer than ten.	Acknowledged as a bias source. We do not train on identity labels, so the bias does not directly affect ranking quality, but it would matter for any future supervised fine-tuning.
Image standardisation	All images are 250×250 RGB JPEG. Pose, lighting, expression vary widely.	A realistic challenge for both CNN and ViT. Justifies CLAHE and Haar Cascade choices below.

Face detection rate	OpenCV Haar Cascade detected faces in approximately 85–90% of images on first pass.	Remaining 10–15% fall back to using the full image. Documented as a limitation in Section 6.
Patch suitability for ViT	After resize to 64×64, faces decompose into 64 non-overlapping 8×8 patches.	Each patch covers approximately one facial feature (eye, nose, mouth region), giving the ViT meaningful tokens.

2.3 Data Cleaning & Feature Engineering

A single preprocessing pipeline was applied identically to both CNN and ViT inputs. Holding preprocessing constant across the two algorithms isolates differences in search quality to the architectures themselves.

Step	Operation	Rationale
1	Face detection — OpenCV Haar Cascade locates the face region.	Strips most of the background; reduces noise the embedding has to absorb.
2	Margin expansion — bounding box expanded by 20% on each side.	Includes hair, ears, neck, all of which carry identity-discriminative information.
3	Resize — crop scaled to 64×64 pixels.	Common input size for both models; gives ViT exactly 64 patches of 8×8.
4	CLAHE — Contrast Limited Adaptive Histogram Equalisation.	Improves contrast in shadowed and unevenly-lit regions without over-amplifying uniform areas.
5	Normalisation — pixel values rescaled from [0, 255] to [0.0, 1.0].	Keeps activations and gradients in numerically stable ranges.
6	Channel reorder — HWC → CHW.	Matches the tensor layout assumed by both networks' first layer.
7	ViT-only — split the 3×64×64 tensor into 64 patches of 3×8×8.	Produces the token sequence consumed by the patch embedding layer.

2.4 Justification of Pre-processing

Three preprocessing decisions deserve specific justification because they materially affect the comparison results.

CLAHE over plain histogram equalisation. Plain histogram equalisation operates globally and over-amplifies contrast in already-uniform regions, generating artefacts in cheek and forehead areas. CLAHE applies the equalisation in small tiles with a clip limit, preserving local contrast where it matters (eye sockets, lip outlines) without amplifying flat regions. We empirically inspected ten randomly chosen images before and after each method and selected CLAHE.

64×64 input size. Three sizes were considered: 32×32, 64×64, and 96×96. 32×32 lost too much detail; 96×96 made the ViT patch grid awkward ($12 \times 12 = 144$ patches, exceeding our memory target). 64×64 strikes the right balance, and it gives ViT exactly 64 patches of 8×8 — a clean, interpretable correspondence between patches and approximate facial features.

Identical pipeline for both models. Any difference in search results between the CNN and ViT can be attributed to the architecture itself rather than data preparation. This is methodologically important for the comparison results in Section 3.5.

3. Product Development and Algorithms

3.1 Algorithm Selection

Two neural network architectures were selected to represent fundamentally different approaches to computer vision: a Convolutional Neural Network (CNN) and a Vision Transformer (ViT). The choice was deliberate. CNNs and Transformers represent the two dominant paradigms in modern computer vision (LeCun et al., 1998; Dosovitskiy et al., 2020), and a pairwise from-scratch comparison demonstrates first-principles understanding of both. We considered and rejected three alternatives: (i) a Siamese network with contrastive loss — would require labelled pairs we do not have; (ii) a pretrained backbone such as FaceNet or ArcFace — would not satisfy the assessment requirement to demonstrate algorithmic understanding; (iii) classical descriptors such as HOG or LBP — well-understood but pedagogically unrewarding given the module's deep-learning focus.

Criterion	CNN (Algorithm 1)	ViT (Algorithm 2)
Year introduced	1998 (LeCun et al.)	2020 (Dosovitskiy et al.)
Core operation	Convolution + max pooling	Multi-head self-attention
Feature scope	Local — sliding 3×3 filter window	Global — every patch attends to every other
Input format	3×64×64 pixel grid	64 patches of 8×8
Parameter count	~500K	~2M
CPU inference (single image)	~200ms	~800ms
Embedding output	128-dim L2-normalised	128-dim L2-normalised
Implementation	NumPy from scratch	NumPy from scratch

3.2 Implementation & Tuning

3.2.1 CNN Architecture

The CNN consists of four convolutional blocks (each: Conv2D → BatchNorm → ReLU → MaxPool), followed by a flatten step and two fully connected layers, terminating in an L2 normalisation layer that produces a 128-dimensional unit vector. Every layer was implemented from scratch in NumPy: ConvLayer (with stride and padding), BatchNormLayer, MaxPoolLayer, DenseLayer, DropoutLayer, and the ReLU activation.

Layer	Output shape	Purpose
-------	--------------	---------

Input	$3 \times 64 \times 64$	RGB face image
Conv2D(16, 3×3) + BN + ReLU + MaxPool	$16 \times 32 \times 32$	Low-level edge / colour features
Conv2D(32, 3×3) + BN + ReLU + MaxPool	$32 \times 16 \times 16$	Mid-level texture features
Conv2D(64, 3×3) + BN + ReLU + MaxPool	$64 \times 8 \times 8$	High-level part features
Conv2D(64, 3×3) + BN + ReLU + MaxPool	$64 \times 4 \times 4$	Deep abstraction
Flatten → Dense(1024 → 256) + ReLU + Dropout(0.3)	256	Compress
Dense(256 → 128)	128	Embedding vector
L2 Normalise	128 (unit vector)	Cosine-ready

Tuning notes — Three architectural choices were tuned. Filter counts were doubled at each block ($16 \rightarrow 32 \rightarrow 64 \rightarrow 64$) following standard CNN practice; we tested $32 \rightarrow 64 \rightarrow 128 \rightarrow 128$ but the parameter count tripled with no observable benefit on visual inspection. Dropout rate was set to 0.3 after testing 0.1, 0.3, and 0.5 — 0.5 caused unstable activation magnitudes; 0.1 was effectively a no-op. Self-similarity (cosine similarity of an image's embedding with itself) was verified at 1.000 as a sanity check on the L2 normalisation step.

3.2.2 ViT Architecture

The ViT splits each $3 \times 64 \times 64$ face image into 64 non-overlapping 8×8 patches, projects each patch linearly into a 128-dimensional embedding, prepends a learnable [CLS] token, adds learnable positional encodings, and passes the resulting 65-token sequence through four Transformer blocks. Each Transformer block contains: LayerNorm → Multi-Head Self-Attention (4 heads, scaled dot-product attention) → residual connection → LayerNorm → MLP (GELU activation, $4 \times$ hidden expansion) → residual connection. After the final block, the [CLS] token is extracted, passed through a final LayerNorm and linear projection, and L2 normalised to produce the 128-dimensional embedding. All components — PatchEmbedding, MultiHeadSelfAttention, MLPBlock, TransformerBlock, LayerNorm, the CLS token, and the positional encoding — were implemented from scratch in NumPy.

Tuning notes — Number of heads was tuned at 2, 4, and 8; 4 was selected as the best speed/expressivity trade-off on CPU (8 heads doubled inference time with marginal observed benefit). Number of Transformer blocks was tuned at 2, 4, and 6; 4 was selected because each additional block roughly doubles inference latency and 6 took ~ 1.5 s per query, breaking the

interactive UX target. The attention output reshape was a real defect during development — the initial implementation produced a tensor of shape (heads, tokens, dim) when the next layer expected (tokens, heads $\times$ dim); this was caught and fixed in Iteration 7 of the logbook.

3.2.3 Similarity Search

Both algorithms produce 128-dimensional L2-normalised embeddings. Cosine similarity reduces to a single dot product between unit vectors, range [0, 1] (or [-1, 1] before clipping). Search is implemented as a single NumPy matrix-vector multiply against the pre-computed database matrix of shape (N_images, 128), followed by argsort. For N = 2,461 images (LFW Top-50), this completes in under 5 milliseconds on a laptop CPU, so search latency is dominated by the embedding extraction step, not the search itself.

3.3 Iterative Product Development

Development proceeded over eleven weekly iterations, with a 5-minute progress presentation to the class group every Thursday from 29 January 2026 onward. The project logbook (submitted as a separate document, FaceTrace_Logbook.pdf) records each week's iteration, the practical impact of changes (parameter tuning, architecture changes, accuracy or latency observations), individual contributions from Anjaly and Surekha, and the GitHub commit history for that week. The eight major iterations are summarised below.

Iteration	Focus	Key result / impact
1	Dataset selection & project setup	LFW Top-50 chosen; class imbalance noted; environment & GitHub repo bootstrapped.
2	Basic UI — upload & display	Flask /api/search endpoint accepts multipart uploads; image displayed back to user.
3	CNN from scratch	Four-block CNN producing 128-d embeddings; self-similarity = 1.000.
4	Database indexing & cosine search	LFW Top-50 indexed; full search pipeline returns ranked results in <500ms.
5	UI refinement	Dark theme, similarity bars, person-grouped results, threshold slider.
6	Internet-wide search investigation	Documented six legal/technical/scale barriers; project pivoted to dual-algorithm depth (Section 4).

7	ViT from scratch	Patch embedding, MHSA (4 heads), CLS token, positional encoding, four Transformer blocks. Self-similarity = 1.000.
8	Comparison interface	compare.html runs both apps simultaneously, computes per-criterion winner, generates summary.

3.4 Integration of Sustainability & Resilience

Sustainability and resilience were not bolt-on requirements; they shaped the algorithmic choices themselves.

- CPU-only inference budget directly constrained the architecture. The 64×64 input size, the 128-d embedding, and the 4-block / 4-head depth budgets were chosen so that both models run interactively (sub-second CNN, ~1s ViT) on a laptop. A larger model would have been more accurate but would have broken the resilience requirement of running on commodity hardware.
- Privacy-by-architecture is enforced through localhost-only binding and zero network egress in the inference path. There is no toggle that could accidentally exfiltrate biometric data to a cloud endpoint.
- Open-stack components (Flask, OpenCV, NumPy) ensure that the system can be maintained by future students without commercial license dependencies, supporting long-term resilience of the deliverable as a teaching artefact.
- Failure isolation: CNN and ViT run as independent Flask services on different ports. If one process crashes, the other remains available. The frontend handles partial failures gracefully (it labels the failed pane and shows the working one).

4. System Architecture & Deployment

4.1 End-to-End Pipeline

The end-to-end pipeline transforms a user-uploaded photograph into a ranked list of similar faces. The same nine-stage flow is executed for both CNN and ViT services; only the embedding extraction step differs.

1. User uploads a photograph through the browser frontend. JavaScript packages the image as a multipart/form-data POST to the /api/search endpoint.
2. Flask backend decodes the image bytes and converts to an RGB NumPy array.
3. OpenCV Haar Cascade locates face regions. If no face is detected, the full image is used as fallback.
4. Face crop is expanded by 20%, resized to 64×64, CLAHE-enhanced, and channel-reordered to CHW.
5. CNN path: four convolutional blocks → flatten → two FC layers → 128-d L2-normalised vector. ViT path: 64 patches → patch embedding → CLS token + positional encoding → four Transformer blocks → CLS extraction → 128-d L2-normalised vector.
6. Cosine similarity is computed against all indexed embeddings using a single matrix-vector multiplication.
7. Results are ranked by similarity, filtered by a user-configurable threshold, and grouped by identity name for clarity.
8. JSON response is returned to the frontend, which renders cards with thumbnails, similarity bars, and source labels.
9. On the comparison page, both services are queried in parallel; results are displayed side-by-side with a per-criterion winner badge.

Component	Technology	Purpose
Algorithm 1	Custom CNN — NumPy	Face embedding (128-dim)
Algorithm 2	Custom ViT — NumPy	Face embedding (128-dim, patch attention)
Face detector	OpenCV Haar Cascade	Locate face in image
Backend	Flask (Python) — two REST servers	/api/search endpoints
Database	NumPy .npy + JSON metadata	Embeddings & filename store

Frontend	HTML / CSS / JavaScript	Upload, results UI, threshold slider
Comparison UI	compare.html	Side-by-side CNN vs ViT
Dataset	LFW Top-50	Face image database

4.2 Prototype / Demo

Two Flask web servers are deployed simultaneously on the same machine. The CNN application is served at `http://localhost:5000` with a blue-themed UI; the ViT application is served at `http://localhost:5001` with a purple-themed UI; the standalone `compare.html` page connects to both APIs and renders side-by-side results with automatic winner detection. Both servers were tested on Windows 11 (HP laptop, Intel CPU, no GPU) and confirmed functional throughout the assessment period. The live demo will be presented in the final session and walks through three reference queries, showing per-query winner detection and a summary of dual-algorithm behaviour.

4.3 Usability

Usability features deliberately included in the frontend: drag-and-drop upload as well as click-to-browse; a similarity-threshold slider with live result re-rendering; person-grouped results so multiple matches of the same identity are visually clustered; an architecture information panel exposing what each model is doing internally; and a clearly labelled colour-coded distinction between CNN (teal) and ViT (violet) results. Edge cases handled gracefully: no face detected (system informs the user, falls back to full-image embedding); corrupt or unsupported image format (400 response with readable error message); empty database (the search endpoint returns a structured empty result rather than crashing).

5. Critical Evaluation & Limitations

5.1 Algorithm Comparison Results

Property	CNN	Vision Transformer	Winner
Inference speed (CPU)	Faster (~200ms)	Slower (~800ms)	CNN
Memory usage	~500K parameters	~2M parameters	CNN
Local textures (edges, lines)	Strong (convolutional filters)	Weaker (attention is global)	CNN
Implementation simplicity	Moderate	High (attention mechanism)	CNN
Architecture novelty	Established (1998)	State of the art (2020)	ViT
Holistic face understanding	Partial (local only)	Yes (every patch attends)	ViT
Feature scope	Local convolution	Global attention	ViT
Overall — face search	Practical and fast	Theoretically superior	Tied

Both models use randomly initialised weights and were not trained on labelled face identity data. Similarity scores therefore reflect structural image properties rather than learned facial identity. A trained model (ArcFace for CNN, DINOv2 for ViT) would amplify the gap between the two architectures and almost certainly favour ViT given recent literature. The honest interpretation of our results is that the comparison demonstrates architectural behaviour, not retrieval accuracy at deployment quality.

5.2 What Worked

- Both CNN and ViT were fully implemented from scratch in NumPy. No ML framework was used. This demonstrates first-principles understanding rather than library glue.
- End-to-end pipeline from image upload to ranked results works reliably for both algorithms across all tested edge cases.
- The comparison interface successfully runs both algorithms simultaneously and identifies the better-performing algorithm per query.
- The ViT implementation correctly realises multi-head self-attention, positional encoding, GELU activation, and the [CLS] token mechanism as described in the original paper.
- Flask REST APIs are clean, well-structured, and handle the documented edge cases (no face detected, corrupt image, empty database).

- Eleven weekly iterations show systematic, controlled improvement from a basic prototype to the dual-algorithm comparison system.

5.3 Limitations

- Neither CNN nor ViT was trained on face data. Without training, embeddings are not semantically meaningful — similarity reflects low-level image statistics rather than facial identity.
- ViT is approximately four times slower than the CNN on CPU due to the $O(n^2)$ complexity of self-attention across 65 tokens.
- LFW Top-50 is a small benchmark. A practical face search system needs millions of indexed images and a vector index (FAISS, ScaNN, or similar) rather than a brute-force NumPy multiply.
- Haar Cascade face detection fails on non-frontal poses (~10–15% fallback rate). Modern detectors (RetinaFace, MTCNN) would address this but were out of scope.
- No formal accuracy metrics (TAR@FAR, ROC, rank-1 identification rate) were computed because untrained weights make these uninformative.

6. Why Internet-Wide Search Was Not Achievable

During Iteration 6, we investigated extending FaceTrace to perform internet-wide reverse face search — the core feature of commercial systems such as PimEyes. This section documents why the attempt was unsuccessful and is treated explicitly because it represents a real engineering decision the project had to navigate.

6.1 What PimEyes Does

PimEyes operates a continuous web crawler that downloads publicly accessible images from across the internet. Each image is processed by a deep neural network to extract a face embedding, which is then stored in a searchable index containing billions of faces. When a user uploads a query, PimEyes performs a nearest-neighbour search across this index. The product is built on years of crawling and tens of millions of dollars of infrastructure investment.

6.2 Why Our Attempt Failed

Barrier	Explanation
No web-scale database	PimEyes has indexed billions of images over years. Our system has only LFW Top-50. An internet search returned no matches simply because we had no internet-sourced images to compare against.
Legal restrictions	Scraping social media (Facebook, Instagram, LinkedIn) violates Terms of Service. Platforms block automated crawlers using CAPTCHAs, rate limiting, and IP banning.
Infrastructure scale	Crawling even 1% of publicly available face images requires terabytes of storage, dozens of servers, and months of crawling. Not feasible on student hardware.
Untrained models	Both CNN and ViT use random initial weights. Without training on face data, embeddings are not semantically meaningful — matching accuracy would be very low even with a large database.
Dynamic web content	Modern websites render via JavaScript. Simple HTTP scraping is insufficient — a full headless browser is required.

6.3 Pivot

Rather than attempting to replicate the scale of PimEyes — years of engineering and millions of euros of infrastructure — the team focused on implementing two correct underlying architectures with genuine technical depth, and demonstrating their practical difference through a live comparison interface. The limitation is data scale and model training, not algorithmic design. This

is documented honestly here so the marker can see that the project boundary was a deliberate, defended choice rather than an oversight.

7. Conclusion

FaceTrace delivers a privacy-preserving reverse face search prototype that meets the assessment requirements: a real-world consumer application, an explicit sustainability and resilience focus, two AI/ML algorithms applied and evaluated, a complete data acquisition and preprocessing pipeline, an end-to-end deployed system, and an iterative development process documented week by week. Both CNN and ViT were implemented from scratch in NumPy without any ML framework, demonstrating algorithmic understanding at the level the module targets. The honest critical evaluation in Section 5 records both strengths (correct from-first-principles implementations, working comparison interface, fully functional pipeline) and limitations (untrained weights, small dataset, no formal accuracy metrics). The internet-wide search analysis in Section 6 documents a deliberate engineering boundary. The accompanying logbook records every weekly iteration, individual contribution, and the team's evaluation of the AI/ML tools used during development.

8. References

- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J. and Houlsby, N. (2020). An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale. International Conference on Learning Representations (ICLR), 2021.
- Deng, J., Guo, J., Xue, N. and Zafeiriou, S. (2019). ArcFace: Additive Angular Margin Loss for Deep Face Recognition. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4690–4699.
- Huang, G.B., Ramesh, M., Berg, T. and Learned-Miller, E. (2007). Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments. University of Massachusetts Amherst, Technical Report 07-49.
- European Parliament and Council (2016). Regulation (EU) 2016/679 (General Data Protection Regulation), Articles 9 and 22 (concerning biometric data and automated decision-making).

Appendix A. Individual Contribution Summary

Contributions are summarised below. The week-by-week breakdown is in the accompanying logbook (FaceTrace_Logbook.pdf, Section 2).

Anjaly	Surekha
CNN architecture design and NumPy implementation (ConvLayer, BatchNorm, MaxPool, Dense, Dropout, ReLU, L2)	Dataset research, sourcing, and exploratory data analysis (LFW Top-50)
Flask REST API for the CNN service (port 5000)	Vision Transformer (ViT) implementation (PatchEmbedding, MHSA, MLP, TransformerBlock, CLS token)
Face database design and cosine similarity search (NumPy .npy + JSON)	Face detection pipeline (OpenCV Haar Cascade, margin expansion, fallback handling)
LFW dataset indexing pipeline	Frontend UI design (CNN blue theme, ViT purple theme)
Comparison interface (compare.html, winner detection logic)	Flask REST API for the ViT service (port 5001)
ArcFace V2 integration research (future-work scoping)	Weekly 5-minute presentations and ongoing documentation